ADZE: Allelic Diversity Analyzer
Version 2.0

Zachary A. Szpiech Mattias Jakobsson Noah A. Rosenberg

Abstract

ADZE implements a rarefaction approach to estimating allelic richness, private allelic richness, and private allelic richness for combinations of populations, correcting for differences in sample size between populations. Version 2.0 computes the same estimators as version 1.0 — results are identical to the last bit — but reads its data in one pass, caches the rarefaction probabilities, parallelizes over loci, and replaces the parameter file with a conventional command-line interface. Section 10 lists everything that changed.

Contents

1 Introduction **2**
- 1.1 Allelic richness . . . 2
- 1.2 Private allelic richness . . . 2
- 1.3 Private allelic richness for a combination of populations . . . 3
- 1.4 Summary statistics . . . 3
- 1.5 One gene copy . . . 3

2 Getting started **4**
- 2.1 Obtaining ADZE . . . 4
- 2.2 Building . . . 4
- 2.3 Running . . . 4

3 Input **5**
- 3.1 Choosing a format . . . 5
- 3.2 STRUCTURE format . . . 5
- 3.3 Dimensions are measured, not declared . . . 6
- 3.4 VCF format . . . 6
- 3.5 Groupings . . . 7
- 3.6 Missing data and locus filtering . . . 7
- 3.7 How a run proceeds . . . 8

4 Parameters and options **8**
- 4.1 Parameter file syntax . . . 8
- 4.2 Reporting a single g . . . 9
- 4.3 Option reference . . . 9
  - 4.3.1 Input options . . . 9
  - 4.3.2 Analysis options . . . 11

4.3.3 Output options . . . 12
4.3.4 Other options . . . 12

5 Output **13**
5.1 Allelic richness and private allelic richness . . . 13
5.2 Private allelic richness of tuples . . . 13
5.3 Locus-specific output . . . 14
5.4 Filtered loci . . . 14
5.5 Run summary . . . 14
5.6 Version 1.0's layout . . . 15
5.7 Exit status . . . 15

6 Sliding windows **15**
6.1 What a windowed statistic is here . . . 15
6.2 Coordinates . . . 15
6.3 Defining the windows . . . 16
6.4 g is fixed across the scan . . . 16
6.5 Output . . . 16

7 Haplotype alleles **17**
7.1 What a haplotype allele is here . . . 17
7.2 Phase is required . . . 17
7.3 One uncalled genotype makes the block uncalled . . . 18
7.4 Richness saturates as blocks lengthen . . . 18
7.5 Naming and output . . . 18
7.6 Haplotype blocks in sliding windows . . . 19

8 Examples **19**
8.1 The distributed example data set . . . 19
8.2 A larger data set . . . 20
8.3 Naming the tuples of interest . . . 20
8.4 A genome scan . . . 24

9 Performance and threading **25**
9.1 Haplotype blocks . . . 25

10 Changes from version 1.0 **26**
10.1 Compatibility . . . 26
10.2 Interface . . . 26
10.3 Behaviour . . . 27
10.4 Verification . . . 28

References **29**

1 Introduction

The analysis of the distributions of alleles across populations is important for elucidating genetic diversity and population relationships. Two fundamental quantities for a population at a given locus are the number of distinct alleles in the population and the number of alleles found in that population and no other. Both are strongly dependent on sample size: a larger sample of gene copies is more likely to contain any given allele, so counts taken from samples of different sizes cannot be compared directly.

Rarefaction addresses this by computing, for a standardized sample size g, the expected number of alleles that *would* be observed if g gene copies were drawn from each population. The estimators ADZE computes are exact expectations under sampling without replacement, not resampling averages, so no random number generator and no seed are involved.

ADZE implements these computations for allelic richness, private allelic richness, and a generalization of private allelic richness to combinations of populations, as described in Szpiech et al. (2008).

1.1 Allelic richness

Consider a locus with I distinct alleles, and define N_{ij} as the number of copies of allele type i in a sample from population j. Let $N_j = \sum_{i=1}^{I} N_{ij}$ be the sample size of population j at the locus. The probability of finding no copies of allele type i in a subsample of g gene copies from population j is

$$Q_{ijg} = \frac{\binom{N_j - N_{ij}}{g}}{\binom{N_j}{g}}. \tag{1}$$

The probability of finding at least one copy of allele type i in a sample of size g from population j is $P_{ijg} = 1 - Q_{ijg}$, and

$$\hat{\alpha}_g^{(j)} = \sum_{i=1}^{I} P_{ijg} \tag{2}$$

is the estimated allelic richness of a sample of size g from population j (Hurlbert, 1971; Petit et al., 1998; Kalinowski, 2004). Equation 2 estimates the expected number of distinct alleles that will be observed in population j in a sample of size g.

1.2 Private allelic richness

Using this notation, the estimated private allelic richness for a sample of size g from population j is

$$\hat{\pi}_g^{(j)} = \sum_{i=1}^{I} \left[P_{ijg} \left(\prod_{\substack{j'=1 \\ j' \neq j}}^{J} Q_{ij'g} \right) \right], \tag{3}$$

where J is the total number of populations (Kalinowski, 2004). This formula sums over distinct allele types i the probability that a random subsample of size g from population j contains allele type i and that subsamples of size g from the remaining populations do not.

1.3 Private allelic richness for a combination of populations

Generalizing the concept of private allelic richness, we can consider the number of distinct alleles private to some combination of k populations selected from $\{1, 2, \ldots, J\}$. Let $S = \{1, 2, \ldots, J\}$ and let C_k be the set of all combinations of k elements from S; there are $\binom{J}{k}$ of them. Labelling these combinations C_{km}, where m ranges from 1 to $\binom{J}{k}$, the estimated number of distinct alleles private to the mth combination of k populations, when samples of size g are drawn from each of the J populations, is

$$\hat{\pi}_{gk}^{(m)} = \sum_{i=1}^{I} \left[\left(\prod_{j \in C_{km}} P_{ijg} \right) \left(\prod_{j' \in S \setminus C_{km}} Q_{ij'g} \right) \right]. \tag{4}$$

Here $S \setminus C_{km}$ denotes the set S excluding the elements of C_{km}. For $k = 1$, $\hat{\pi}_{gk}^{(m)}$ reduces to private allelic richness as in Equation 3. For $k = J - 1$, Equation 4 can be considered a measure of "missing allelic richness", and it reduces to

$$\hat{\mu}_{g}^{(j)} = \sum_{i=1}^{I} \left[Q_{ijg} \left(\prod_{\substack{j'=1 \\ j' \neq j}}^{J} P_{ij'g} \right) \right], \tag{5}$$

the expected number of alleles absent from population j but present in every other population.

1.4 Summary statistics

ADZE evaluates Equations 2–4 at each of many loci in a data set, and its primary method of reporting results gives the mean, variance, and standard error of the mean across these loci. Locus-specific estimates can also be written out (section 5.3). The mean across loci is calculated at each g and for each population or population grouping as

$$\bar{x} = \frac{1}{L} \sum_{\ell=1}^{L} x_\ell, \tag{6}$$

where L is the number of loci used in the calculation and x_ℓ is one of the statistics above evaluated at locus ℓ. The variance across loci is

$$s^2 = \frac{1}{L-1} \sum_{\ell=1}^{L} (x_\ell - \bar{x})^2, \tag{7}$$

and the standard error of the mean is

$$\mathrm{SE} = \frac{s}{\sqrt{L}}. \tag{8}$$

A statistic is reported at a given g only if it is defined at that g for every locus used, which requires $g \leq N_j$ at each locus for the grouping concerned. With a single locus ($L = 1$) the variance and standard error are undefined and are reported as `nan`.

1.5 One gene copy

The sweep starts at $g = 1$, which is worth reading once because each statistic means something different there and one of them is a constant.

Allelic richness at $g = 1$ is exactly 1 at every locus: with $Q_{ji1} = (N_j - N_{ji})/N_j$ each allele contributes its own frequency p_{ji}, and the frequencies sum to 1. Drawing a single gene copy always yields one distinct allele, so the row is the estimator agreeing with a certainty rather than reporting anything about the data. It is still useful as a check: a locus that does not give 1 means the allele binning or the recurrence is wrong. Its variance across loci comes out as a number around 10^{-33}, the square of the rounding in a sum of ones, which is numerically zero.

Private allelic richness at $g = 1$ is $\sum_i p_{ji} \prod_{j' \neq j} (1 - p_{j'i})$: the chance that a single gene copy drawn from j carries an allele that single draws from every other grouping all miss. Note that every allele present in j contributes, weighted by how likely it is to be missed elsewhere — this is not the summed frequency of the alleles strictly private to j. The tuple statistic at $g = 1$ is the same form with the product of p taken over the tuple's members.

2 Getting started

2.1 Obtaining adze

Source is at `https://github.com/szpiech/ADZE`. Version 2.0 requires nothing but a C++17 compiler. Two libraries are used when the toolchain provides them, and the program builds and runs without either: OpenMP for `--threads`, and zlib for compressed input. Without zlib a `.gz` file is refused with a message rather than misread.

2.2 Building

With CMake:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
cmake --install build --prefix ~/.local      # optional
```

Or with plain `make`:

```
make -C src                   # serial, no compressed input
make -C src OPENMP=1 ZLIB=1  # GCC, or clang on Linux
make -C src OPENMP=1 ZLIB=1 OMPFLAGS="-Xpreprocessor -fopenmp" OMPLIBS=-lomp
                              # Apple clang, with libomp installed
```

CMake reports at configure time which of the two it found; the `make` build takes each as a flag. Without OpenMP, `--threads` is accepted but warns that it is being ignored.

2.3 Running

There are two ways to specify a run, and they can be mixed:

```
adze --data mydata.stru --group-col 5 --out-prefix mypops
adze myparamfile.txt --tolerance 0.05
```

The first form takes everything from the command line. The second reads a parameter file in the version 1.0 format — the same keywords, and the same short flags — with command-line options taking precedence over the file. Three sources are consulted, in increasing order of precedence: built-in defaults, then the parameter file if one is given, then the command line.

`--help` lists every option; `--version` prints the version and the method reference. Both work with no data file and no parameter file. `--write-template` writes a commented parameter file that runs as it stands once `DATA_FILE` is filled in.

Only the data file is strictly required. Everything else either has a default or is measured from the data (section 3.3).

3 Input

3.1 Choosing a format

Two input formats are read: the STRUCTURE-like layout that ADZE has always taken (section 3.2), and VCF (section 3.4), optionally compressed with `gzip` or `bgzip`. `--format` selects one; its default, `auto`, reads the file name — anything ending `.vcf`, after an optional `.gz` or `.bgz`, is treated as VCF.

The two paths converge immediately: both reduce each locus to per-grouping allele counts and missing-data tallies, which is all the estimators use, and both lay those counts out identically for the same genotypes. Results therefore do not depend on which format the data arrived in — the same genotypes written both ways produce byte-identical output, which `test/formats.py` checks.

3.2 structure format

This file is expected in the same format used by STRUCTURE: one row per gene copy, so a diploid individual occupies two consecutive rows, preceded by a header row of locus names. Each data row begins with one or more label columns and continues with one allele code per locus. Allele codes are arbitrary tokens — integers, as in the example below, or any other strings that do not contain whitespace.

Figure 1 shows `example/small_data.stru`, distributed with ADZE: five loci, three groupings, and eleven diploid individuals, with missing alleles coded `-9`.

```
D12S1638 D14S1007 D9S1779 D9S1825 D7S2477
1037 87 Pima Mexico AMERICA 126 128 124 -9 142
1037 87 Pima Mexico AMERICA 126 128 142 -9 142
1038 87 Pima Mexico AMERICA 120 124 124 129 142
1038 87 Pima Mexico AMERICA 126 128 152 137 142
   ...
1357 24 Basque France EUROPE 126 120 124 -9 154
1357 24 Basque France EUROPE 128 122 126 -9 156
   ...
747 684 Japanese Japan EAST_ASIA 126 -9 124 129 158
747 684 Japanese Japan EAST_ASIA 128 -9 144 135 158
   ...
```

Figure 1: The distributed example data file, abridged. Row 1 holds the locus names; each subsequent row is one gene copy, with five label columns followed by five allele codes.

3.3 Dimensions are measured, not declared

Version 1.0 required `LOCI`, `DATA_LINES`, `NON_DATA_ROWS` and `NON_DATA_COLS` to be supplied by hand, and validated them against the file. Version 2.0 measures them while reading:

`--loci`
from the number of names in the header row.

`--data-lines`
by counting data rows.

`--non-data-cols`
from the width of the first data row minus the locus count.

`--group-col`
defaults to the last label column.

`--non-data-rows`
defaults to 1, the locus-name row. This is the one value that cannot be inferred, so give it explicitly if the file has additional header rows.

`--max-g`
defaults to the largest standardized sample size the data supports, which is the smallest N_j over all groupings and all *surviving* loci — resolved after missing-data filtering, since dropping loci can raise it.

Any of these may still be given, in which case the declared value is used and a warning is printed if it disagrees with what was measured. `--dry-run` reports the detected layout, the per-grouping sample sizes, the largest feasible `MAX_G` and the number of tuples an analysis would evaluate, then exits without computing anything.

3.4 VCF format

Each record is one locus and each allele index is one allele type, so the reference allele is allele 0 and the alternates follow. A sample contributes as many gene copies as its `GT` field holds, which lets haploid and diploid records sit in the same file, and a `.` is an uncalled copy. A sample's ploidy is the largest number of alleles any of its `GT` fields holds, since that is how many gene copies it could have contributed; a record where it carries fewer — a haploid call in an otherwise diploid file, say — counts the difference as missing, and so tells against that locus under `--tolerance`. Only `GT` is read: no other `FORMAT` field, and no `INFO`, `QUAL` or `FILTER` value, affects the counts, so filtering must be done before ADZE sees the file. Locus names come from the `ID` column, or from `CHROM:POS` where `ID` is `.`.

A VCF names its samples but says nothing about which population each belongs to, so `--samples` is required with VCF input. It takes a file of two whitespace-separated columns, sample name then grouping name:

```
# sample        grouping
HG00096         GBR
HG00097         GBR
NA18939         JPT
```

Anything after # is a comment and any further columns are ignored, so an existing sample sheet can usually be passed as it stands. Groupings are numbered in the order they first appear in this file, which is the order they appear in the output. A sample in the VCF that the map does not mention is skipped, and a sample the map names that the VCF does not contain is ignored; both are reported as warnings with a count.

Because a sample's gene copies are what the missing-data fraction is measured against, `--tolerance` means the same thing here as for STRUCTURE input: the fraction of a grouping's gene copies that are not called at that locus (section 3.6).

```
adze --data cohort.vcf.gz --samples populations.tsv \
     --tolerance 0.05 --out-prefix cohort
```

`NON_DATA_ROWS`, `NON_DATA_COLS` and `GROUP_BY_COL` describe the STRUCTURE layout and are ignored, with a warning, for VCF input.

3.5 Groupings

With STRUCTURE input, rows are grouped by the value in the `--group-col` label column; with VCF input the grouping comes from `--samples`. In both cases groupings appear in output in the order they are first seen; any label column can be used, so a file annotated with population, region and continent can be analysed at any of those levels without being rewritten, and the same VCF can be analysed at several levels by swapping the map file. `--pops` restricts the analysis to a comma-separated list of groupings and `--exclude-pops` removes them; both change J, and therefore change what "private" means, so they are a way of asking a different question rather than a way of subsetting output.

3.6 Missing data and locus filtering

An allele code equal to `--missing` (default `-9`) is treated as unobserved and reduces N_j at that locus. Because a statistic is reported at a given g only when it is defined at every locus, one locus with a small N_j limits the whole analysis.

`--tolerance` x drops any locus at which some grouping has more than a fraction x of its gene copies missing. Loci dropped this way are listed in the `_deletedloci` file (section 5.4). Filtering is a trade: it discards loci to buy a larger `MAX_G`, and section 8.1 works through the trade-off on the example data.

The default is $x = 0.1$. Version 1.0's default, $x = 1$, keeps every locus, and on modern data that is rarely what is wanted: a single locus at which one grouping scored nothing drives `MAX_G` to 1 and leaves that grouping undefined at every g, so a genome-wide run can complete and report nothing. `--tolerance 1` restores the old behaviour for anyone who wants it. A parameter file that declares `TOLERANCE` is unaffected, and 1.0's files all declare it.

Two conditions make a grouping unusable, and both are now reported by name rather than left to be inferred from empty output. A locus at which no allele is observed in any grouping contributes nothing: such loci are counted and reported, and treated as undefined at every g. A locus at which one particular grouping scored fewer than g gene copies makes that grouping undefined at g — and because the sentinel propagates through the average, one such locus is enough. ADZE therefore reports, per grouping, the fewest copies it scored anywhere, how many loci sit at that floor, and how many it did not score at all, adding an explicit remedy when the floor leaves nothing usable. `--dry-run` reports the same ceiling without computing anything. Under `--legacy` such a grouping

is absent from the result file entirely, which is what the warning is there to explain. If filtering removes every locus, ADZE stops with an error rather than reporting statistics over no data.

3.7 How a run proceeds

Version 2.0 reads the data twice and holds none of it.

The first pass counts. It reads every gene copy and records, for each grouping and each locus, how many copies were scored there — one integer per locus per grouping, and nothing about which alleles they were. That is enough to decide everything the analysis needs to know about the dataset as a whole: which loci fail the missing-data filter (the filter's denominator is a grouping's total, which is only final at the end of the file), how large g can be for each grouping and overall, where the windows fall, and what a `--dry-run` should report.

The second pass computes. Loci arrive one at a time; for each, the non-detection probabilities Q_{jig} are built once and every requested statistic reads them — allelic richness, private allelic richness, and each tuple. Each statistic keeps a running mean and sum of squared deviations per g, and one per window per g, so nothing per-locus is stored: when a locus's rows have been written it is forgotten.

Peak memory therefore does not follow the genotypes. On a VCF of 320 000 loci and 10 groupings, version 1.0's successor-in-design — the same program with the dataset resident — peaks at 195 MB where 2.0 peaks at 32 MB; the gap widens with the number of loci.

A `--dry-run` answers from the first pass alone and never opens the data a second time, so asking what a large file would do no longer costs most of doing it.

structure input is converted first

A STRUCTURE file is individual-major: one row per gene copy, one column per locus, so no locus can be finished without reading every row. Rather than a second engine, such a file is converted once into locus-major counts — a chunk of locus columns at a time, sized to a memory budget — and the analysis reads those. The converted file holds counts rather than genotypes and is correspondingly small: an 80 000-locus file of 122 MB converts to 7.2 MB. It is written beside the output and removed when the run ends, unless `--count-file` names it, in which case it is kept and reused by the next run over the same data. A kept file records where it came from — the data file's path, size and modification time, the grouping filters, the missing code and the column layout — and is refused with the reason if any of those has changed, rather than being used with a warning. `--convert-chunk` sets the loci per conversion pass.

The cost is honest: a STRUCTURE run reads its input twice and its conversion once, so it is slower than it was (on 320 000 loci, 4.1 s against 5.7 s) while using roughly half the memory. A VCF run pays only the extra scan.

4 Parameters and options

4.1 Parameter file syntax

A parameter file is a sequence of lines, each a keyword followed by its value:

```
DATA_FILE small_data.stru
GROUP_BY_COL 5
MAX_G 8
```

```
TOLERANCE 0.24
# everything after a hash is a comment
```

Anything after `#` is ignored. The value is the rest of the line with surrounding whitespace removed, so values may contain spaces and may contain other keywords as substrings. Boolean parameters take `0` or `1`.

4.2 Reporting a single g

By default every g from 1 to `MAX_G` is reported. `--at-g` asks for one instead:

`--at-g` ***N***
: report only $g = N$.

`--at-g max`
: report only at whatever `MAX_G` resolves to, without having to know the number in advance.

One flag governs every output: the three statistics, their per-locus companions and their window files all carry the single g, with the same columns as a full run — including the g column — so a consumer reads either the same way.

Two ceilings bind the request: `MAX_G`, which is what the run was asked to sweep, and the smallest number of gene copies scored anywhere, which is the largest g with defined values. A request above the lower of the two is met at that ceiling, with a warning naming the value asked for, the value used and which bound applied. This differs deliberately from an over-large `MAX_G`, which keeps its shape and reports the unreachable rows as undefined: a ladder with `NA` at the top is still a ladder, whereas a single-g report of nothing but `NA` would be no answer at all. A value below 1 is refused rather than rounded up, g being a sample size.

Because the recurrence in g is sequential, the sweep still has to climb to the g being reported — but no further, and the buffers hold no more than that. The saving therefore scales with how far below `MAX_G` the request sits: on 20 000 loci in 12 groupings with all 66 pairwise tuples and `MAX_G` at 33, `--at-g 2` ran in 0.58 s against 6.72 s for the full ladder, while `--at-g 33` saved 3%, the climb being the whole ladder anyway.

4.3 Option reference

The following is generated from the option table in the source, so it lists exactly what the program accepts. Each entry gives the long option, the version 1.0 short flag where one exists, the parameter-file keyword where one exists, and the default the program applies when the option is not given. That default is declared once, in the same table, and is what `--help` prints and what the README tabulates; `test/docs.py` fails if the README and the table disagree about a value.

4.3.1 Input options

`--data` *FILE*, `-f`, `DATA_FILE` **(paramfile)**
: *string*
genotype file: STRUCTURE layout or VCF, optionally gzipped (required)

`--format` *FMT*, `FORMAT` **(paramfile)**
: *string*
input format: auto, structure or vcf; auto reads the file extension *Default:* `auto`.

`--samples` *FILE*, `SAMPLE_FILE` **(paramfile)**
string
sample-to-grouping map, required for VCF input (two columns: sample grouping)

`--loci-map` *FILE*, `LOCI_MAP` **(paramfile)**
string
locus coordinates for STRUCTURE input (three columns: locus chromosome position)

`--count-file` *FILE*, `COUNT_FILE` **(paramfile)**
string
keep the counts a STRUCTURE file is converted into, and reuse them next run *Default:* `temporary, removed`.

`--convert-chunk` *N*, `CONVERT_CHUNK` **(paramfile)**
int
loci per pass when converting STRUCTURE input *Default:* `as many as fit 64 MB`.

`--max-g` *N*, `-g`, `MAX_G` **(paramfile)**
int
largest standardized sample size *Default:* `the largest the data supports`.

`--non-data-rows` *N*, `-nr`, `NON_DATA_ROWS` **(paramfile)**
int
header rows before the genotypes *Default:* `1`.

`--non-data-cols` *N*, `-nc`, `NON_DATA_COLS` **(paramfile)**
int
label columns before the genotypes *Default:* `detected`.

`--loci` *N*, `-l`, `LOCI` **(paramfile)**
int
number of loci *Default:* `detected`.

`--data-lines` *N*, `-d`, `DATA_LINES` **(paramfile)**
int
number of data rows *Default:* `detected`.

`--group-col` *N*, `-s`, `GROUP_BY_COL` **(paramfile)**
int
which label column names the grouping *Default:* `the last label column`.

`--missing` *STR*, `-m`, `MISSING` **(paramfile)**
string
code for a missing allele *Default:* `-9`.

`--pops` *LIST*
string
analyse only these groupings (comma-separated) *Default:* `every grouping`.

`--exclude-pops` *LIST*
string
analyse everything except these groupings *Default:* `none excluded`.

4.3.2 Analysis options

`--hap-snps` *N*, `HAP_SNPS` **(paramfile)**
int
collapse runs of N consecutive SNPs into haplotype alleles (phased VCF only) *Default:* `off`.

`--hap-bp` *N*, `HAP_BP` **(paramfile)**
int
collapse each N-basepair run of SNPs into haplotype alleles (phased VCF only) *Default:* `off`.

`--min-hap-snps` *N*, `MIN_HAP_SNPS` **(paramfile)**
int
do not report a haplotype block holding fewer than N SNPs *Default:* `1`.

`--window-bp` *N*, `WINDOW_BP` **(paramfile)**
int
sliding windows N basepairs wide (needs locus coordinates) *Default:* `off`.

`--window-loci` *N*, `WINDOW_LOCI` **(paramfile)**
int
sliding windows N loci wide (surviving loci; needs coordinates for chromosomes) *Default:* `off`.

`--step-bp` *N*, `STEP_BP` **(paramfile)**
int
advance basepair windows by N; a step equal to the width means no overlap *Default:* `the window width`.

`--step-loci` *N*, `STEP_LOCI` **(paramfile)**
int
advance locus windows by N loci *Default:* `the window width`.

`--min-window-loci` *N*, `MIN_WINDOW_LOCI` **(paramfile)**
int
do not report a window holding fewer than N loci *Default:* `1`.

`--at-g` *N*/*max*, `AT_G` **(paramfile)**
string
report only this g, or max for whatever MAX_G resolves to *Default:* `every g`.

`--stat` *LIST*
string
which statistics to compute: richness,private,tuples *Default:* `richness,private`.

`--tolerance` *X*, `-t`, `TOLERANCE` **(paramfile)**
float
drop a locus if any grouping exceeds this missing fraction; 1 keeps every locus *Default:* `0.1`.

`--combinations`, `-c`, `COMB` **(paramfile)**
boolean
also compute private alleles of grouping tuples *Default:* `off`.

`--tuples-k` *LIST*, `-k`, `K_RANGE` **(paramfile)**
string
tuple sizes for –combinations, e.g. 2 or 1-3 or 1,3,5-7

`--tuples` *FILE*
string
compute only the named tuples in FILE (one per line) instead of every k-subset *Default:* `every k-subset`.

`--threads` *N*
int
worker threads; requires an OpenMP build *Default:* `1`.

4.3.3 Output options

`--out-prefix` *PREFIX*
string
compose output names from PREFIX *Default:* `adze`.

`--out-richness` *FILE*, `-r`, `R_OUT` **(paramfile)**
string
allelic richness output file *Default:* `from --out-prefix`.

`--out-private` *FILE*, `-p`, `P_OUT` **(paramfile)**
string
private allelic richness output file *Default:* `from --out-prefix`.

`--out-tuples` *FILE*, `-o`, `C_OUT` **(paramfile)**
string
tuple output file (one per tuple size, suffixed _k) *Default:* `from --out-prefix`.

`--full-richness`, `-fr`, `FULL_R` **(paramfile)**
boolean
also write per-locus allelic richness *Default:* `off`.

`--full-private`, `-fp`, `FULL_P` **(paramfile)**
boolean
also write per-locus private allelic richness *Default:* `off`.

`--full-tuples`, `-fc`, `FULL_C` **(paramfile)**
boolean
also write per-locus tuple values *Default:* `off`.

`--legacy`, `LEGACY_FORMAT` **(paramfile)**
boolean
write version 1.0's layout: space-separated, no header, undefined rows omitted *Default:* `off`.

4.3.4 Other options

`--params` *FILE*
string
read parameters from FILE (also accepted as the first argument)

`--dry-run`
boolean
report the detected layout, groupings and feasible MAX_G, then stop *Default:* `off`.

`--progress`, `-pp`, `PRINT_PROGRESS` **(paramfile)**
boolean
progress bars *Default:* `on when stderr is a terminal`.

`--quiet`
boolean
suppress progress and informational messages *Default:* `off`.

`--tolerance-only`, `-tnocalc`, `TNC` **(paramfile)**
boolean
apply the missing-data filter, report it, and stop *Default:* `off`.

`--no-check`, `-skipchk`, `SKIP_CHK` **(paramfile)**
boolean
accepted for compatibility; validation is now free and always on *Default:* `off`.

5 Output

By default ADZE writes `adze.richness`, `adze.private` and `adze.summary.txt` in the current directory; `--out-prefix` replaces the `adze` stem. Giving an explicit name with `--out-richness`, `--out-private` or `--out-tuples` uses that name literally and appends version 1.0's suffixes (`_summary`, `_fulldata`, `_deletedloci`) to it.

Results go to files; progress, warnings and errors go to standard error, so redirecting standard output captures results alone. `--quiet` silences the diagnostics. Progress bars are drawn only when standard error is a terminal.

5.1 Allelic richness and private allelic richness

One row per grouping per g: the grouping name, the g at which the statistics were computed, the number of loci used, and then the mean, variance and standard error across those loci (Equations 6–8). The columns are named on the first line and separated by tabs, and a statistic undefined at that g is written as `NA`.

```
POP_GROUPING    G   NUM_LOCI    MEAN        VAR         STD_ERR
AMERICA         1   3           1           0           0
AMERICA         2   3           1.34524     0.0973639   0.180152
```

`--legacy` writes version 1.0's layout instead (section 5.6).

5.2 Private allelic richness of tuples

The same layout, with the tuple's grouping names in place of the single name. With `--tuples-k` the tuple size k is appended to the file name, one file per k; with `--tuples` the named tuples all go to one file.

5.3 Locus-specific output

`--full-richness`, `--full-private` and `--full-tuples` add a `_fulldata` file carrying the statistic at every locus: one row per locus and grouping, one column per g, tab-separated with a header. A value undefined at that g is `NA`.

```
LOCUS   POP_GROUPING    G1      G2      G3
L0      POP1            1       1.53571 1.80357
L0      POP2            1       1.48214 1.75
L1      POP1            1       1.60714 2
L1      POP2            1       1.55357 1.91071
```

The locus leads the row because the program reads one locus at a time and computes every grouping's value for it before moving on (section 3.7): a locus's rows are written together and never returned to.

This is the transpose of what version 1.0 wrote, and the one output format that 2.0 deliberately breaks. 1.0 put a row per grouping and g with a column per locus, the locus names in the header and the summary appended at the end of each row:

```
POP_GROUPING G NUM_LOCI L0 L1 L2 L3 MEAN VAR STD_ERR
POP1 2 4 1.53571 1.60714 1 1.67857 1.45536 0.095557 0.154561
```

A file as wide as the dataset is unreadable by ordinary tools at genome scale — a million loci gave a handful of rows of a million fields, past the column limit of every spreadsheet and most parsers — and it cannot be written until every locus has been computed, which is what forced the whole ladder of per-locus values to be held in memory (section 10). One row per locus streams as the sweep runs, and joins to a locus table the way a reader expects. The layout is the same under `--legacy`, which governs the statistics files; the summary columns are not repeated here, being the statistics file's business. To recover 1.0's orientation, transpose it:

```
awk -F'\t' 'NR>1 {print $1, $2, $3}' out.richness_fulldata    # locus, label, g=1
```

In the example above `POP1` scored only two gene copies at `L2`, so its richness there is undefined at $g = 3$ and the whole grouping is undefined at that g; that locus is undefined from $g = 3$ up, and the summary columns with it. Version 1.0 dropped the whole row as soon as one locus was undefined, so a grouping with a single badly scored locus disappeared from the file at every g with nothing to say why; `--legacy` reproduces that.

5.4 Filtered loci

When `--tolerance` is below 1, a `_deletedloci` file records how many loci were dropped, the threshold applied, and the name of each dropped locus, in the order they appear in the input. (Version 1.0 listed them in descending order of position; the change is a consequence of naming each locus as it is read rather than collecting the names first.) The file is written even when every locus is dropped — the run that produces no statistics at all is the one whose user most needs the list.

5.5 Run summary

A `_summary` (or `.summary.txt`) file is written on every run. It echoes the parameters actually used — including values that were measured rather than declared, and `MAX_G` when it was resolved from the data — and records the elapsed time at which each phase finished.

5.6 Version 1.0's layout

The result files are tab-separated with a header row, and a statistic that is undefined at a given g is written as `NA`.

`--legacy` asks for version 1.0's layout: no header, space separators, groupings separated by a blank line, and rows that are undefined at a given g omitted entirely rather than marked. That omission is why the default changed. A consumer of the legacy format cannot distinguish "not computable here" from "not run", and a grouping that is undefined at every g — which one badly scored locus is enough to cause (section 3.6) — disappears from the file altogether, leaving a run that completed and reported nothing with no visible explanation. There is one flag and no way to ask for both layouts at once.

Tuple labels are comma-joined in the default layout and space-joined in the legacy one, so under tabs a tuple stays a single column.

The window files (section 6) have always been tab-separated with a header, whichever layout is in force: they are new in 2.0 and have no earlier consumer to stay compatible with. The `_fulldata`, `_deletedloci` and `_summary` files are likewise unaffected by the flag — and note that `_fulldata` is transposed with respect to 1.0 in both modes (section 5.3), so `--legacy` does not reproduce that file.

5.7 Exit status

0	success
2	usage error: unknown option, missing value, contradictory request
3	I/O error: a file could not be opened, read or written
4	data error: the file disagrees with itself or with the parameters

6 Sliding windows

Every statistic in section 1 can be reported window by window along the genome instead of, or as well as, once for the whole dataset.

6.1 What a windowed statistic is here

For each window, ADZE takes the loci inside it and reports the same mean, variance and standard error over those loci that it would report over all of them. Nothing about the estimators changes: $\hat{\alpha}_{jg}$ and $\hat{\pi}_{jg}$ are expectations for a single locus, and a window is simply a different set to average over.

Two consequences are worth stating plainly. The statistic has no haplotype or linkage component — it does not know that the loci in a window are near each other, only that they were grouped together — so a window here is a smoothing device, not a measure of local haplotype structure. (For alleles that are haplotypes, see section 7; the two combine, a windowed haplotype run averaging over the blocks in each window.) And the variance and standard error are taken across the loci in the window, so with few loci they are noisy; with one locus the variance is undefined and ADZE writes `nan`. Use `--min-window-loci` to set a floor.

6.2 Coordinates

A window is a range of the genome, so every locus has to be placed on it. With VCF input the coordinates come from `CHROM` and `POS`. The STRUCTURE layout carries none, so `--loci-map` supplies

them:

```
# locus   chromosome   position
L0        chr1         5000
L1        chr1         10000
L2        chr2         5000
```

Whitespace-separated, `#` starts a comment, further columns are ignored. Every locus in the data must appear: a window over loci whose positions are unknown would be a window in name only, so a missing coordinate stops the run rather than producing a quietly meaningless scan. Loci must arrive in genome order — positions increasing within a chromosome, and each chromosome in one block — and ADZE says which locus breaks the order rather than sorting a dataset it is streaming.

6.3 Defining the windows

A window is measured either in basepairs or in loci, never both:

`--window-bp` ***N***
: windows N basepairs wide, anchored at position 1 and advancing by `--step-bp`. The intervals do not depend on where the loci fall, so the same region gives the same windows in every run and two datasets can be compared window by window. How many loci a window holds varies with locus density.

`--window-loci` ***N***
: windows of N surviving loci, advancing by `--step-loci`. Each window carries the same weight in the averaging; its width in basepairs varies with density. The interval reported is the span of the loci it holds.

Each step defaults to its window width, so the default is a non-overlapping tiling; a step smaller than the width is the sliding case. No window crosses a chromosome. Windows are laid out over the loci that survive the missing-data filter (section 3.6), so a locus dropped there is absent from every window, and a locus-counted window of N means N surviving loci.

`--min-window-loci` N suppresses windows holding fewer than N loci; the default of 1 reports every window that holds anything. Windows holding no loci at all are never reported.

6.4 g is fixed across the scan

`MAX_G` is resolved genome-wide, as it is for the whole-dataset statistics: the largest standardized sample size every grouping supports at every surviving locus. It is not re-resolved per window. A window in a sample-rich region is therefore not reported at a larger g than the rest of the scan, which is what makes windows comparable with each other — the point of a scan. Since a set containing a locus that cannot reach g is undefined at that g, this also means that under the default every window is defined at every reported g. If you raise `MAX_G` past what some locus supports, the windows containing such loci are written as `NA` rather than dropped, so the file keeps one row per window per g.

6.5 Output

Each requested statistic writes an extra file named by appending `_windows`: with `--out-prefix scan`, `scan.richness` gains `scan.richness_windows`. The whole-dataset files are written exactly as before; asking for windows never changes them.

Window files always carry a header and are always tab-separated, whatever `--legacy` asks for — they are new output with no version 1.0 consumer to stay compatible with, and a scan is read by machine. The columns are

```
CHROM  START  END  POP_GROUPING  G  NUM_LOCI  MEAN  VAR  STD_ERR
```

with `TUPLE` in place of `POP_GROUPING` in the tuple file, where the grouping names are joined with commas so the label stays one field. Rows are ordered by grouping, then by g, then along the genome, so the track for one grouping at one g is a contiguous block.

A scan is usually read at one g, and `--at-g` (section 4.2) is the economical way to ask for it: the file holds one row per window per grouping instead of one per window per grouping per g. On the scan below that is the difference between 912 rows and 48.

Windowed tuple output is large: one row per window per g per tuple. `--dry-run` reports the row count each statistic would write, along with the window count and how many windows fell below the threshold, before anything is computed.

7 Haplotype alleles

The statistics of section 1 count alleles at a locus. Given phased data they can count *haplotypes* across a short run of loci instead: `--hap-snps` and `--hap-bp` collapse consecutive SNPs into blocks, and a block's alleles are the haplotypes its gene copies carry.

```
adze --data cohort.vcf.gz --samples pops.tsv --hap-snps 5 --out-prefix hap
adze --data cohort.vcf.gz --samples pops.tsv --hap-bp 10000 --out-prefix hap
```

Nothing about the estimators changes. $\hat{\alpha}_{jg}$ and $\hat{\pi}_{jg}$ are written as sums over the alleles a locus carries, and a block is a locus whose alleles happen to be haplotypes; the rarefaction, the averaging and the output are the ones described everywhere else in this manual. What changes is what an allele *is*, and that brings its own assumptions.

7.1 What a haplotype allele is here

For each gene copy, ADZE reads its allele at every SNP in the block, in order along the chromosome, and treats that sequence as one allele. Two gene copies carry the same haplotype allele when they match at every SNP in the block. Blocks are disjoint: consecutive SNPs are collapsed, not slid over, so each SNP belongs to exactly one block and no gene copy is counted twice.

`--hap-snps` N makes each block N consecutive SNPs, so every block carries the same number of markers while its width in basepairs varies with marker density. `--hap-bp` N makes each block an N-basepair span, anchored at position 1 and advancing by N, so the same region yields the same blocks in every run and two datasets can be compared block by block; there the number of SNPs varies instead. A block never spans a chromosome, a span holding no SNP produces no block, and `--min-hap-snps` drops a block that holds too few.

7.2 Phase is required

A haplotype is a run of alleles along one chromosome copy, so the input must say which copy each allele sits on. ADZE therefore requires phased genotypes: every multi-copy `GT` must use `|`, and a record that uses `/` stops the run, naming the record and the sample. Haploid copies are trivially

phased and need nothing. Phasing is not something ADZE does or checks the quality of; a haplotype assembled from an unphased genotype would be an arbitrary pairing presented as a measurement, which is why this is an error and not a warning.

For the same reason the STRUCTURE layout is not supported in this mode: it has no way to say which copy an allele belongs to. Supplying `--hap-snps` with STRUCTURE input is refused as a usage error.

The records must also be sorted, by chromosome and then position. A block is a run of *consecutive* records, so out of order they would join SNPs that are not neighbours and report the result as a haplotype. An ordinary run needs sorted input only for sliding windows; a haplotype run needs it always, and stops at the first record that goes backwards or returns to a chromosome it has left (`bcftools sort` fixes either).

7.3 One uncalled genotype makes the block uncalled

A gene copy carries a haplotype allele in a block only if it is called at *every* SNP in the block. One uncalled genotype anywhere in the block and that copy contributes nothing there; it is not given a partial haplotype, which would be an allele no copy actually carries and would inflate every count that read it. A copy whose `GT` is shorter than the sample's ploidy at some record counts as uncalled there too.

The consequence is worth sizing before a run rather than discovering afterwards: missing data compounds along the block. If each SNP is called independently with probability $1 - m$, a copy survives a block of N SNPs with probability about $(1 - m)^N$, so at 2% missingness a block of 50 SNPs keeps roughly a third of its gene copies. Since N_j enters the rarefaction directly, that shows up as a smaller g the data can support, not as a quietly different answer. `--tolerance` then applies to blocks exactly as it applies to loci — a block where some grouping has lost more than the allowed fraction of its gene copies is dropped — and the `_blocks` file (section 7.5) reports the callable copies per grouping for every block, so the cost is a number rather than an inference.

7.4 Richness saturates as blocks lengthen

A long block gives every gene copy its own haplotype. When a block carries as many distinct haplotypes as it has callable copies, no haplotype repeats, every draw of g copies yields g distinct haplotypes, and $\hat{\alpha}_{jg} = g$ whatever the data says: the statistic has stopped measuring anything. ADZE counts such blocks and says so at the end of the run, and the `HAPLOTYPES` column of the `_blocks` file is where the approach to saturation can be watched.

Private allelic richness degrades differently and more gently: a long haplotype is more likely to be private to a grouping simply because it is long, so $\hat{\pi}_{jg}$ grows with block length for reasons that have nothing to do with population history. Both effects argue for the shortest blocks that answer the question, and for comparing block lengths with each other rather than reading one in isolation.

7.5 Naming and output

A block spanning more than one SNP is named `chromosome:start-end`; a block holding exactly one SNP takes that record's own name, so `--hap-snps 1` reproduces an ordinary SNP-level run exactly. Every output file is the one described in section 5, with blocks where loci would be — including the per-locus `_fulldata` files and, in a windowed run, the window files, a block being assigned to a window by its start position.

One file is added. `_blocks` describes the blocks themselves:

```
BLOCK             CHROM  START  END    SNPS  HAPLOTYPES  KEPT  POP0  POP1
chr1:1000-4000    chr1   1000   4000   4     14          no    9     5
chr1:5000-8000    chr1   5000   8000   4     11          yes   12    8
```

one row per block in file order, with the gene copies each grouping could score and whether the filter kept it. Two runs can report the same mean richness over blocks of five haplotypes and blocks of fifty, so this file is what makes the difference visible.

7.6 Haplotype blocks in sliding windows

The two combine: a windowed haplotype run averages over the blocks in each window, exactly as an ordinary windowed run averages over loci (section 6), and `--window-loci` counts blocks. A block is placed by its first SNP. Two consequences follow.

With `--window-loci`, a window's reported interval runs from its first block's first SNP to its last block's *last* SNP, so it is the span of the data the window actually holds.

With `--window-bp`, a block belongs to the window containing its start, and can run past that window's end; its haplotypes then include SNPs outside the window's reported interval. ADZE counts such blocks and says so. They disappear when the two grids nest: blocks from `--hap-bp` h fall wholly inside basepair windows whenever h divides both `--window-bp` and `--step-bp`, both grids being anchored at position 1.

```
adze --data cohort.vcf.gz --samples pops.tsv --hap-bp 5000 \
     --window-bp 100000 --step-bp 25000 --out-prefix scan
```

Blocks from `--hap-snps` have no fixed width, so they cannot be made to nest; at typical marker densities a block is short against a window and only those at a window's edge are affected.

`--dry-run` reports the layout a given width produces — the blocks it yields, the SNPs they hold and how wide they came out — from the counting pass alone, which is the cheapest way to size `--hap-snps` before committing to a scan.

8 Examples

8.1 The distributed example data set

The data set in Figure 1 has five loci and three groupings. Running it with nothing but the data file and the grouping column,

```
adze --data small_data.stru --group-col 5 --out-prefix small
```

reports on standard error

```
Read 22 gene copies at 5 loci in 3 groupings (d:h:m:s 0:0:0:0.00)
2 loci have at least one grouping with more than 10% missing data.
MAX_G not given; using 6, the largest the 3 surviving loci support.
```

Two of the five loci exceed the default tolerance of 0.1 and are dropped, and the analysis runs to $g = 6$ over the three that remain. Asking instead for every locus,

```
adze --data small_data.stru --group-col 5 --out-prefix keep --tolerance 1
```

```
MAX_G not given; using 3, the largest the 5 surviving loci support.
```

halves the standardized sample size. Three is low for 22 gene copies, and the reason is missing data rather than sample size: the `EUROPE` grouping has six gene copies in total, but at locus `D9S1825` three of those six are missing, leaving $N_j = 3$. `D9S1825` is also missing 25% of the `AMERICA` grouping, and `D14S1007` is missing 25% of `EAST_ASIA` — which is why the default drops exactly those two.

Which run to prefer is a judgement about the data: the second uses all five loci but compares populations only at $g \leq 3$; the first compares them over a much wider range of g using three loci. The default leans towards the larger g, on the grounds that a locus one grouping barely scored constrains every grouping.

The parameter file distributed with the example, `small_paramfile.txt`, runs the same analysis in version 1.0's style, declaring all four dimensions along with `MAX_G 8` and `TOLERANCE 0.2`:

```
adze small_paramfile.txt
```

Its results agree with the run above wherever the two overlap, and the private allelic richness files are identical. The richness files are not: declaring `MAX_G 8` reports `AMERICA` and `EAST_ASIA` at $g = 7$ and 8, which their own sample sizes reach, whereas the default stops at the largest g that *every* grouping supports. Private allelic richness stops at 6 in both, because no allele can be private at a g that some grouping cannot reach.

8.2 A larger data set

The next example uses the data of Rosenberg et al. (2002): 1056 individuals from 52 populations typed at 377 microsatellite loci, available at `https://rosenberglab.stanford.edu/diversity.html`. Because computing private alleles for all combinations at the population level would be prohibitive, the published analysis groups the individuals by region instead, which is column 5 of that file's labels and reduces the number of groupings from 52 to 7.

The version 1.0 parameter file for this analysis (Figure 3 of the version 1.0 manual) declared `MAX_G 70`, `DATA_LINES 2112`, `LOCI 377`, `NON_DATA_ROWS 1`, `NON_DATA_COLS 5`, `GROUP_BY_COL 5`, `K_RANGE 2-6` and `TOLERANCE 1`. In version 2.0 the four dimensions are measured from the file, so the same run is

```
adze --data diversitydata.stru --group-col 5 --max-g 70 \
     --stat richness,private,tuples --tuples-k 2-6 \
     --out-prefix hgdp --threads 8
```

Figures 2–4 show the published analysis of these data: mean allelic richness against g, mean private allelic richness against g, and mean private allelic richness for combinations of six of the seven regions — that is, "missing" allelic richness, Equation 5. These figures are reproduced from the version 1.0 manual and were produced with version 1.0; version 2.0 computes the same values.

Africa carries both the highest allelic richness and, by a wide margin, the highest private allelic richness, while the Americas and Oceania have the highest missing allelic richness — the alleles absent there but present everywhere else.

8.3 Naming the tuples of interest

A full sweep over tuple sizes evaluates $\binom{J}{k}$ combinations for each k, which is 2^J tuples if every k is requested. When only a few groupings matter, list them in a file, one tuple per line:

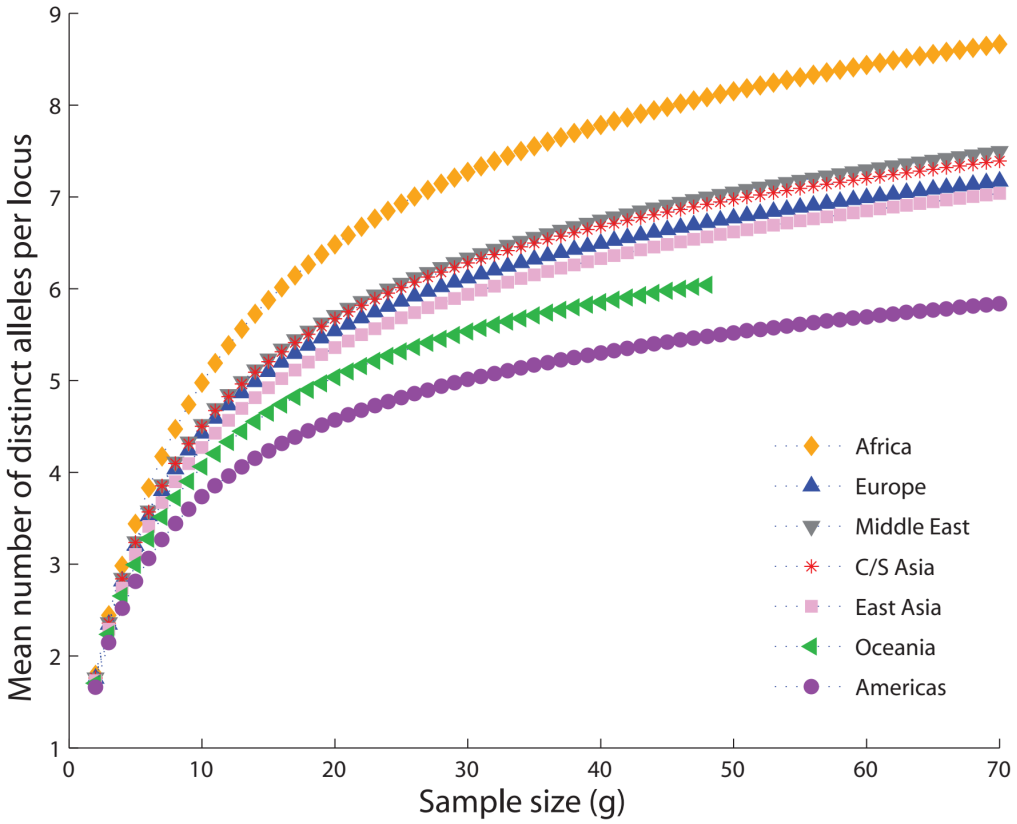


Figure 2: The mean number of distinct alleles per locus as a function of standardized sample size for seven major geographic regions.

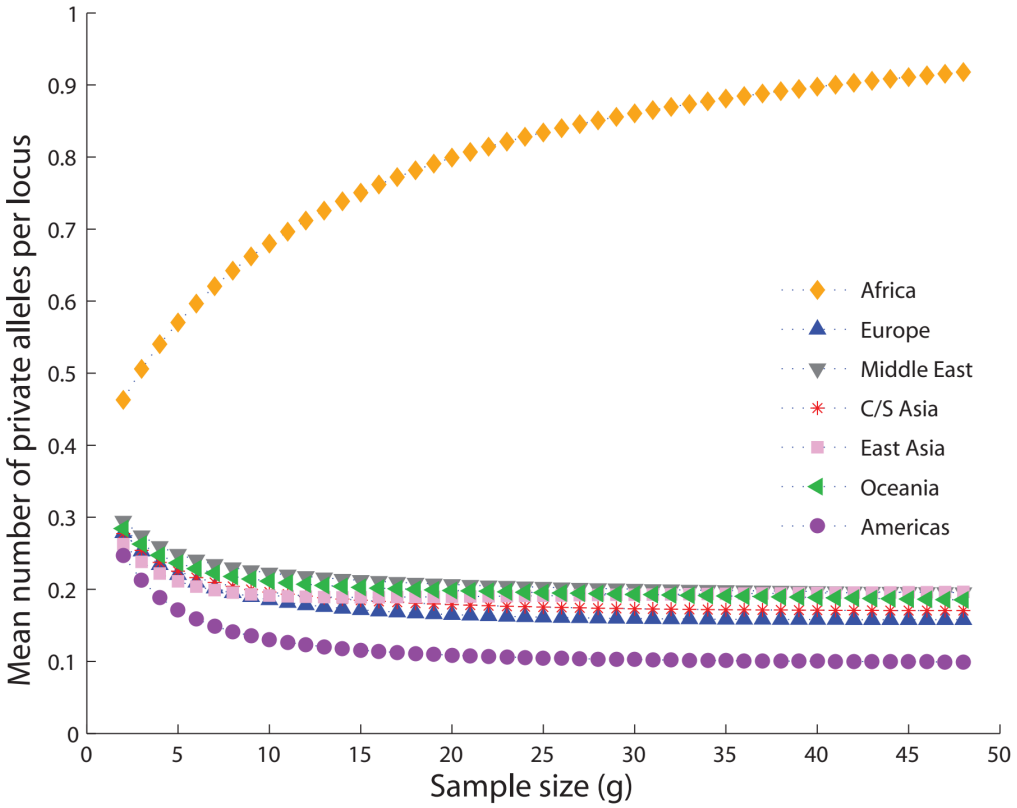


Figure 3: The mean number of private alleles per locus as a function of standardized sample size for seven major geographic regions.

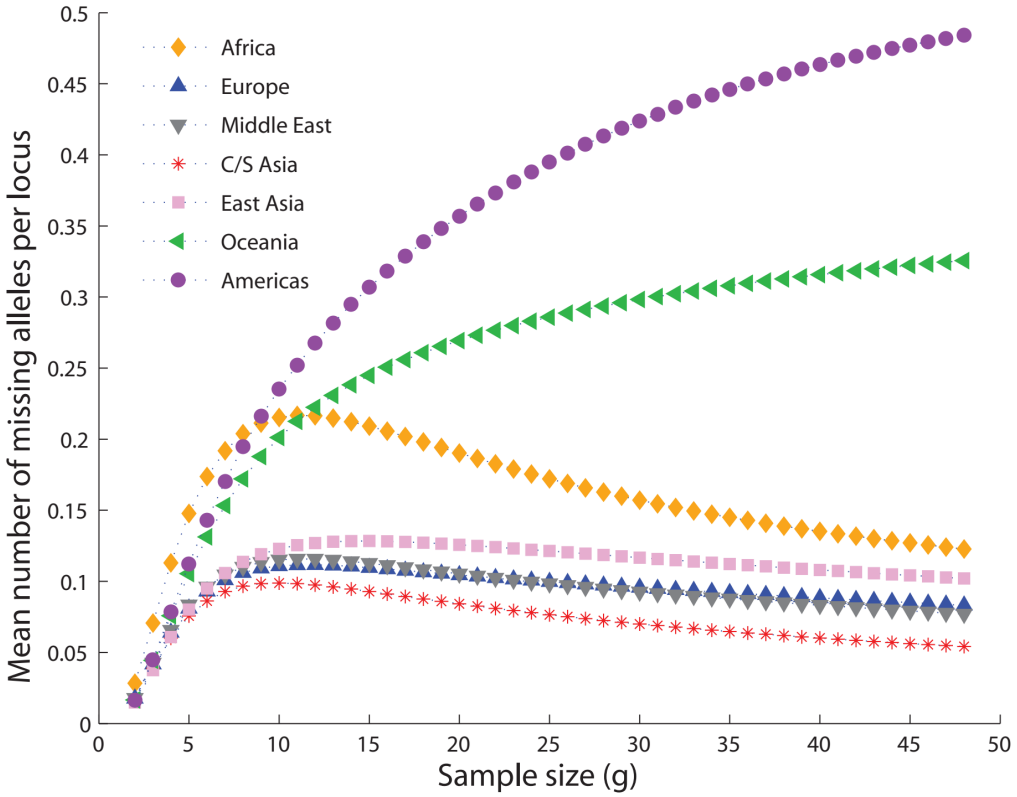


Figure 4: The mean number of missing alleles per locus for seven major geographic regions (private to combinations of six of the seven regions) as a function of standardized sample size.

```
# tuples.txt
AMERICA, EUROPE
EAST_ASIA
AMERICA, EUROPE, EAST_ASIA

adze --data small_data.stru --group-col 5 --tuples tuples.txt \
     --stat tuples --out-prefix regions
```

Each line is a comma- or space-separated list of grouping names as they appear in the data file; `#` starts a comment. This computes only the tuples named, at any mixture of sizes.

8.4 A genome scan

The generator that drives the test suite writes coordinates as well as genotypes, so a scan can be reproduced from the repository alone. This makes 60 loci spaced 5 kb apart over two chromosomes, in four groupings of twelve diploid individuals:

```
python3 -c "import sys; sys.path.insert(0,'test'); import gen_data
gen_data.write_pair('scan', npops=4, nind=12, nloci=60, nall=6,
                    missing=0.03, seed=3, chroms=2, spacing=5000)"
```

Ask what a 50 kb window stepping 25 kb would cover before running it:

```
adze --data scan.vcf --samples scan.samples \
     --window-bp 50000 --step-bp 25000 --dry-run
```

```
Largest feasible MAX_G: 20
MAX_G to be used:       the largest feasible
  (TOLERANCE 0.1 will drop loci first, which can raise both numbers)
Windows:          12 of 50000 bp, step 25000
  loci per window: 4-10
  richness rows:   960
  private rows:    960
```

Twelve windows and, before filtering, g running to 20, every grouping having 24 gene copies and the sparsest locus carrying 20. The projection is deliberately conservative about its own arithmetic: filtering happens first and raises both the reachable g and the row count, as the run itself then reports. Then run it, with a floor on how thin a window may be:

```
adze --data scan.vcf --samples scan.samples \
     --window-bp 50000 --step-bp 25000 --min-window-loci 3 \
     --out-prefix scan
```

```
  7 loci have at least one grouping with more than 10% missing data.
  Laid out 12 windows over 60 loci.
  MAX_G not given; using 22, the largest the 53 surviving loci support.
```

The filter drops seven of the sixty loci, which raises the reachable g from 20 to 22 and the output to 1056 rows — more than a scan is usually read at. Adding `--at-g 10` asks for one g and gives 48, one row per window per grouping:

CHROM	START	END	POP_GROUPING	G	NUM_LOCI	MEAN	VAR	STD_ERR
chr1	1	50000	POP0	10	8	3.45471	0.0842792	0.10264
chr1	25001	75000	POP0	10	8	3.56664	0.0605046	0.0869659

The consecutive windows overlap by 25 kb, so each interior locus contributes to two of them. `scan.richness` holds the whole-dataset figures unchanged alongside, and the rows here are the same bytes the full ladder gives at $g = 10$.

9 Performance and threading

The per-locus loops of all three statistics can be spread over threads with `--threads` in an OpenMP build. Loci are processed independently and reduced in locus order, so the output does not depend on the thread count: the result files are bit-identical at any `--threads` setting.

Compressed input is decompressed as it streams, so a `.vcf.gz` costs more processor time to read than the same data uncompressed, and usually less wall-clock time on a busy filesystem, since there is far less to transfer.

Table 1 gives the effect of the version 2.0 work on the phases that dominate large runs, measured against a version 1.0 build on the same inputs. The costs that grew fastest in version 1.0 were locus filtering (quadratic in the number of loci) and the sweep over g (quadratic in `MAX_G`); both are now linear.

Phase	Condition	Version 1.0	Version 2.0
Locus filtering	8000 loci, 4623 dropped	5.97 s	<0.01 s
Richness + private	1000 loci, `MAX_G` 80	4.87 s	0.24 s
Tuples	12 groupings, 495 tuples	4.33 s	0.67 s
Reading	64 MB input	3.62 s	0.70 s
Peak memory	64 MB input	404 MiB	21 MiB
Statistics, 8 threads	2000 loci, `MAX_G` 150	—	4.0×

Table 1: Measured on an 8-core machine, best of two runs. Version 1.0 held every genotype in memory, so its footprint grew with sample size; version 2.0 keeps allele counts only, and its footprint is flat in the number of gene copies.

9.1 Haplotype blocks

Collapsing SNPs into blocks changes what a run costs in two directions at once, and the balance depends on the block width. The counting pass keeps one count per block per grouping, so a wider block means fewer of them and less memory; the sweep's work at a block grows with the number of distinct haplotypes it carries, which rises with width until every callable copy is unique and then stops. Measured on 40 000 phased biallelic SNPs, 300 gene copies in 6 groupings, 2% of calls missing, `--max-g` 10:

`--hap-snps`	time	peak	blocks	haplotypes per block	callable copies	saturated blocks
off	0.29 s	7.4 MB	40 000	2.0	100%	—
1	0.45 s	8.1 MB	40 000	2.0	98%	0%
2	0.37 s	6.7 MB	20 000	4.0	96%	0%
5	0.45 s	5.6 MB	8 000	31.9	90%	0%
10	0.92 s	5.7 MB	4 000	210.7	82%	0%
20	0.45 s	5.5 MB	2 000	200.2	67%	96%
50	0.26 s	5.4 MB	800	109.4	37%	100%

Three things in that table matter more than the timings. The callable-copy column follows $(1-m)^N$ almost exactly — 98, 96, 90, 82, 67 and 37 % against 98, 96, 90, 82, 67 and 36 predicted — so the all-or-nothing rule of section 7 behaves as the arithmetic says: only a third of the gene copies are still callable at 50 SNPs with 2% missingness, so two thirds of them have been lost. The time is not monotone: it peaks in the middle, where haplotype diversity is rising but blocks are still numerous, and falls again once every copy is unique and there are few blocks left. And the last column is the one to read first: by 20 SNPs almost every block has as many haplotypes as callable copies, where $\hat{\alpha}_{jg}$ is g by construction. The cheapest runs in that table are the ones that measure the least.

10 Changes from version 1.0

10.1 Compatibility

Parameter files written for version 1.0 work unchanged, as do the short flags. Two defaults differ. `--tolerance` is 0.1 rather than 1, so a run that names no tolerance now drops loci that version 1.0 kept; every 1.0 parameter file declares `TOLERANCE` and is therefore unaffected, and `--tolerance 1` restores the old behaviour explicitly (section 3.6). And the result files are tab-separated with a header rather than in 1.0's layout, which `--legacy` restores (section 5.6). A script that parsed 1.0's output needs that one flag, and the differential test suite passes it for exactly that reason. Explicitly named output files keep version 1.0's literal suffixes, and the default result format is unchanged. `SKIP_CHK` and `--no-check` are still accepted but do nothing, because validation now costs nothing: it happens during the single read pass.

10.2 Interface

- Options can be given without a parameter file, and `--help` and `--version` work with no arguments at all. A bare invocation prints usage; version 1.0 wrote a template parameter file into the working directory.
- The parameter-file parser reads a keyword and the rest of the line. Version 1.0 searched each line for any keyword as a substring, so a value that contained a keyword — `R_OUT out_LOCI_richness` — was rejected, and values could not contain spaces.
- Dimensions are measured rather than declared (section 3.3).
- Diagnostics and progress go to standard error; exit status distinguishes usage, I/O and data errors.

- VCF input, plain or `gzip`/`bgzip` compressed, via `--format` and `--samples` (section 3.4). Version 1.0 read only the uncompressed STRUCTURE layout.
- Given phased VCF input, consecutive SNPs can be collapsed into haplotype blocks and the statistics computed over haplotypes rather than single-SNP alleles (section 7).
- The data is never held in memory. Two passes replace the old read: one counts gene copies and decides the filter, the ceilings, `MAX_G` and the windows; the second computes every statistic from one Q table per locus (section 3.7). On a VCF of 1.28 million loci and 10 groupings, peak memory falls from 764 MB to 145 MB at equal run time; a STRUCTURE file of the same size falls from 886 MB to 260 MB and takes about twice as long, being converted to locus-major counts first.
- `--dry-run` is answered from the counting pass and never reads the data a second time: 54 MB and 0.47 s become 11 MB and 0.19 s on 80 000 loci.
- The `_deletedloci` list is in input order rather than descending order of position, and is written even when every locus is dropped (section 5.4).
- Each statistic is still announced, but they are computed together, so the run reports one completion and the summary carries one `Statistics completed at` line rather than one per statistic.
- The `_fulldata` files are transposed: one row per locus and locus with a column per g, where 1.0 wrote a row per g with a column per locus (section 5.3). This is the one output format 2.0 does not reproduce, in either mode.
- Result files are tab-separated with a header and `NA` for undefined rows (section 5.6), and the `_fulldata` files mark an undefined locus `NA` rather than dropping the row (section 5.3). `--legacy` writes version 1.0's layout, which is byte-identical to what 1.0 wrote apart from the $g = 1$ row.
- The `--tolerance` default is 0.1 rather than 1 (section 3.6), and the groupings that a retained locus leaves unusable are named, with their locus counts, instead of showing up as empty result files.
- Every statistic is now also reported at $g = 1$ (section 1.5); version 1.0's sweeps started at $g = 2$. The default output therefore carries one extra row per grouping, which is the one respect in which it is not what 1.0 wrote.
- `--at-g` reports a single g instead of the whole ladder (section 4.2), clamping an over-large request to the largest reachable g with a warning.
- Sliding-window statistics (section 6): `--window-bp` or `--window-loci` with a matching step, `--min-window-loci`, and `--loci-map` to give STRUCTURE input the coordinates a window needs. Version 1.0 reported each statistic once for the whole dataset.
- New: `--out-prefix`, `--stat`, `--pops`, `--exclude-pops`, `--tuples`, `--legacy`, `--dry-run`, `--threads`, `--quiet`.

10.3 Behaviour

Estimates are unchanged. The following differ, in each case because version 1.0 gave an answer that could not be used:

1. A locus with no observed allele in any grouping is reported as undefined, with a warning. Version 1.0 stopped with a segmentation fault.

2. Filtering that removes every locus is an error. Version 1.0 continued and printed `nan` rows over zero loci.

3. A data row whose column count is wrong is always an error. With checking disabled, version 1.0 would misread it silently.

4. The variance of a single locus is reported as `nan` rather than the result of dividing by zero.

5. Reported times are elapsed wall-clock times with two decimals. Version 1.0 reported processor time truncated to whole seconds, which is why the example run in the version 1.0 manual reports `0:0:0:0` throughout.

10.4 Verification

Every change was checked with a differential test that runs version 2.0 and a version 1.0 build over the same inputs and compares their output: nine synthetic data sets — spanning allele-count range, missingness, a single locus, a single grouping, unequal sample sizes, haploid data, and a locus with no observed alleles — crossed with nine parameter settings, plus the distributed example. All 69 comparable cases agree; the remaining cases are those in which version 1.0 crashes or reports statistics over zero loci, and so has no output to compare. The test is in `test/regress.py` and needs a version 1.0 binary to compare against.

What "agree" means has been stated more precisely twice, and both qualifications are deliberate.

- Version 1.0 cannot produce a $g = 1$ row, so whole files do not match. Every row at $g \geq 2$ is compared, and the $g = 1$ rows are accounted for separately: each results file must carry exactly one per grouping or tuple, so a build that quietly stopped writing them fails.

- The per-locus values, the labels and the locus counts are compared exactly — for `_fulldata`, cell by cell, the two layouts being transposes of each other (section 5.3). The three summary columns are compared to a tolerance of 10^{-9} relative, with values below 10^{-12} treated as zero. Version 2.0 forms the mean and variance in one pass as it sweeps, rather than summing twice over stored per-locus values, so the last bits of a variance can differ: on the distributed example, version 1.0 reports a variance of 2.96×10^{-31} where every deviation is zero and 2.0 reports 0. The tolerance is four orders of magnitude tighter than the six significant digits that are printed, so a disagreement in any printed digit still fails the test.

The windowed statistics have no version 1.0 counterpart either, so `test/windows.py` checks them against the layout they claim to cover: one window over a single-chromosome dataset must reproduce the whole-dataset output exactly, every window's mean and variance must match a recomputation from the per-locus columns, the windows must tile as asked and stay inside a chromosome, basepair and locus windows must agree on evenly spaced loci, and the inputs a window cannot be defined over must be refused.

VCF input has no version 1.0 counterpart, so it is checked differently: `test/formats.py` writes one set of genotypes as STRUCTURE, as VCF, and as compressed VCF, and requires the three runs to agree byte-for-byte. Because both readers order allele slots by the same rule, the comparison is exact rather than approximate — any difference at all is a real one, not a change in the order of a floating-point sum.

References

Hurlbert, S. H. 1971. The nonconcept of species diversity: a critique and alternative parameters. *Ecology* 52: 577–586.

Kalinowski, S. T. 2004. Counting alleles with rarefaction: private alleles and hierarchical sampling designs. *Conservation Genetics* 5: 539–543.

Kalinowski, S. T. 2005. HP-RARE 1.0: a computer program for performing rarefaction on measures of allelic richness. *Molecular Ecology Notes* 5: 187–189.

Petit, R. J., A. El Mousadik, and O. Pons. 1998. Identifying populations for conservation on the basis of genetic markers. *Conservation Biology* 12: 844–855.

Rosenberg, N. A., J. K. Pritchard, J. L. Weber, H. M. Cann, K. K. Kidd, L. A. Zhivotovsky, and M. W. Feldman. 2002. Genetic structure of human populations. *Science* 298: 2381–2385.

Szpiech, Z. A., M. Jakobsson, and N. A. Rosenberg. 2008. ADZE: a rarefaction approach for counting alleles private to combinations of populations. *Bioinformatics* 24: 2498–2504.